Benchmark-Aufgabe 05
Ben-Eater-8-Bit-Rechner

Originalgetreuer Nachbau des 8-Bit-Breadboard-Computers mit UART-Lader – Aufgabe, Prüfgrundlagen, Vorbereitung der Hardwarestrecke und Vergleich MAS gegen einen einzelnen OpenCode-Aufruf

Stand	13.10.2026
Aufgabe	`Schaltungsagent/benchmark/prompts/05_ben_eater_8bit.md`
Referenz-Testbench	`Schaltungsagent/benchmark/reference/05_ben_eater_8bit_ref_tb.vhd`
Ergebnisse	`docs/benchmark/ergebnisse/20261013-05_ben_eater_8bit/`
Modell	opencode/muse-spark-1.3-contributor-free (beide Varianten)
Zielhardware	Digilent Arty S7-50, Vivado 2026.1, USB-UART 115 200 Baud

> **Ergebnis in Kürze.** Das MAS setzte den Rechner mit zwölf Modulen ohne Reparatur um und bestand als erster Lauf des Projekts einen **automatischen Hardwaretest am Board**: vier Programme wurden über die UART in den geflashten Rechner geladen, alle Antworten stimmten (45:44 min). Ein einzelner OpenCode-Aufruf mit demselben Modell löste die Aufgabe in 8,4 Minuten ebenso korrekt. Vor den Läufen deckte die Vorbereitung drei Fehler der Hardwarestrecke auf (#58–#60), ohne deren Behebung kein Hardwaretest möglich gewesen wäre.

Inhalt

1. Ziel und Einordnung
2. Aufgabenstellung
3. Programme und erwartetes Verhalten
4. Prüfgrundlagen
5. Vorbereitung der Hardwarestrecke
6. Versuchsaufbau
7. Ergebnisse
8. Unschärfe beim Verwerfen von Laderahmen
9. Einordnung
10. Grenzen
11. Ablage und Reproduktion

1 Ziel und Einordnung

Aufgabe 04 (4-Bit-Akkumulator-CPU) zeigte, dass das MAS eine kleine, frei spezifizierte CPU bis aufs Board bringt. Aufgabe 05 ist der nächste Schritt: ein **bereits existierendes, gut dokumentiertes System**, damit die Entwurfsqualität an einem Original gemessen werden kann. Gewählt wurde der 8-Bit-Breadboard-Computer von Ben Eater, eine weit verbreitete Umsetzung der SAP-1-Architektur aus Malvino/Brown. Er hat einen gemeinsamen 8-Bit-Bus, Register A und B, eine ALU mit Carry- und Zero-Flag, 16 Byte RAM, ein Befehlsregister, einen Befehlszähler, ein Ausgaberegister und ein mikroprogrammiertes Steuerwerk mit 16 Steuersignalen und fünf Mikroschritten je Befehl.

Vorgegeben wurde die Endversion mit Flags-Register und bedingten Sprüngen, **originalgetreu**: Architektur, Steuersignale und Mikroprogramm folgen Ben Eaters veröffentlichtem Mikrocode (`microcode-eeprom-with-flags`). Weil das Repository keine Lizenz angibt, beschreibt die Aufgabe die Tabelle mit eigenen Worten und Quellenangabe, ohne Code zu übernehmen. Angepasst wurde nur, was auf dem FPGA anders sein muss: Takterzeugung, Programmierung, Anzeige. Programme lassen sich zusätzlich über die USB-UART laden – im Protokoll des Hardwaretests des Systems, damit erstmals ein **automatischer Hardwaretest** möglich ist.

2 Aufgabenstellung

2.1 Architektur

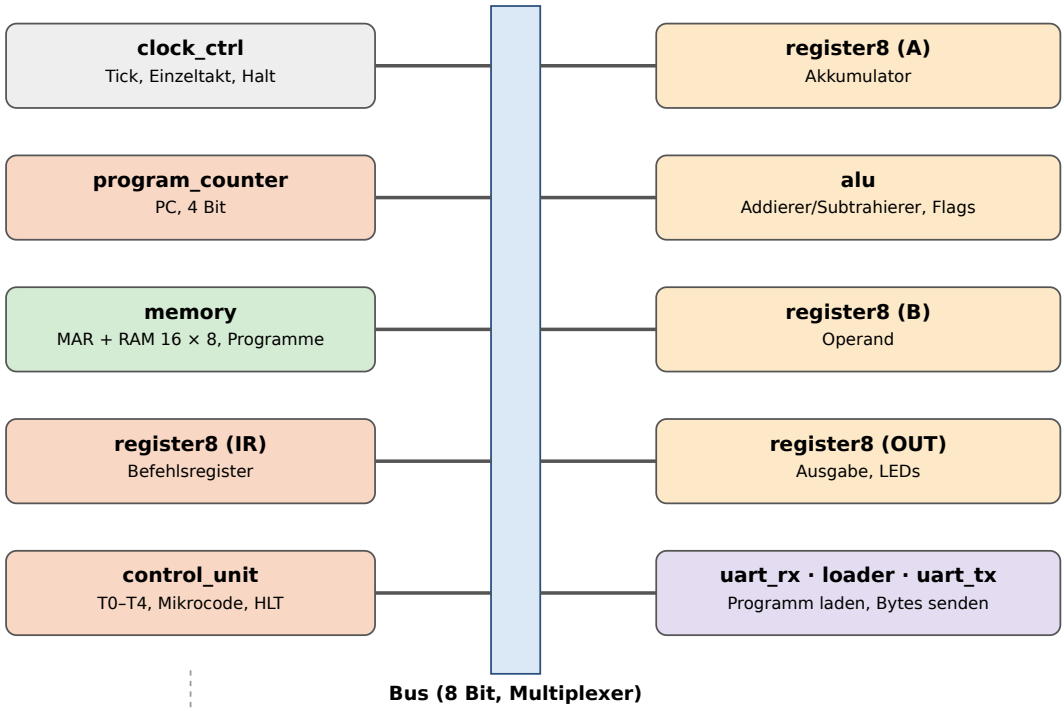


Abbildung 1: Baugruppen und Bus wie im Original-Aufbau (links Steuer- und Speicherseite, rechts Rechenwerk und Ausgabe), mit den Modulnamen der Aufgabe. Den Bus treibt genau die im Steuerwort aktive Quelle (CO, RO, IO, AO, EO), sonst ist er 0.

2.2 Steuersignale und Mikroprogramm

Signal	Wirkung	Signal	Wirkung
HLT	Takt anhalten	EO	ALU-Ergebnis auf den Bus
MI	MAR := Bus(3..0)	SU	ALU subtrahiert
RI	RAM[MAR] := Bus	BI	B := Bus
RO	RAM[MAR] auf den Bus	OI	OUT := Bus
IO	IR(3..0) auf den Bus	CE	PC := PC + 1
II	IR := Bus	CO	PC auf den Bus
AI	A := Bus	J	PC := Bus(3..0)
AO	A auf den Bus	FI	Flags := C, Z der ALU

Alle Befehle holen in T0 mit `MI CO` und in T1 mit `RO II CE`; danach:

Befehl	Opcode	T2	T3	T4
NOP	0000			
LDA a	0001	IO MI	RO AI	
ADD a	0010	IO MI	RO BI	EO AI FI
SUB a	0011	IO MI	RO BI	EO AI SU FI
STA a	0100	IO MI	AO RI	
LDI n	0101	IO AI		
JMP t	0110	IO J		
JC t	0111	IO J, wenn C = 1		
JZ t	1000	IO J, wenn Z = 1		
OUT	1110	AO OI		
HLT	1111	HLT		

Opcodes 1001–1101 wirken wie NOP. Ein Mikroschritt wird mit der Taktflanke ausgeführt, an der die Schrittfreigabe aktiv ist; alle Register übernehmen gleichzeitig, Bus und ALU arbeiten mit den Werten vor der Flanke. Nach T4 folgt T0. HLT ändert kein Register und hält die CPU bis zum Reset oder bis ein Programm über die UART geladen wird. Die ALU addiert bzw. bildet A + (not B) + 1; das Carry ist der Übertrag aus Bit 7 (bei SUB also „kein Borgen“), Zero zeigt ein Ergebnis von 0.

2.3 Schnittstelle, Takt und UART

Port / Generic	Board	Bedeutung
clk	R2, 100 MHz	Systemtakt
rst	BTN0	synchroner Reset; lädt das mit SW3/SW2 gewählte Programm
btn_step	BTN1	Einzeltakt: ein Mikroschritt je Druck (entprellt)
sw(0)	SW0	LED-Anzeige: unteres bzw. oberes Halbbyte von OUT
sw(1)	SW1	Takt: automatisch (TICK_CYCLES, Standard 25 Mikroschritte/s) oder Einzeltakt
sw(3 downto 2)	SW3, SW2	festes Programm beim Reset
led(3 downto 0)	LD5 … LD2	Halbbyte von OUT
uart_rx / uart_tx	V12 / R12	USB-UART 8N1, CLKS_PER_BIT (Standard 868 = 115 200 Baud)

Ladeprotokoll (wie der Hardwaretest des Systems sendet): 4 Bytes Länge n (little endian, 1..16), n Programmbytes, dann „START“ und Zeilenvorschub. Danach steht das Programm im RAM (Rest 0), alle Register sind 0, und die CPU läuft im **UART-Modus** mit einem Mikroschritt je Takt; jedes OUT sendet den Wert als Byte zurück (ein OUT wartet, bis der Sender frei ist). Im Board-Modus sendet die CPU nichts. Der UART-Modus endet nur mit dem Reset; ein neuer Rahmen hält die CPU an, bis er vollständig ist.

2.4 Module

Zwölf Entities, je eine Datei: `input_sync`, `debouncer`, `clock_ctrl`, `program_counter`, `register8` (A, B, OUT, IR), `alu` (mit Flags), `memory` (MAR, RAM, feste Programme), `control_unit` (Mikroschrittzähler, Mikrocode, Halt), `uart_rx`, `uart_tx`, `loader` und das Top-Level `ben_eater_cpu` mit dem Bus-Multiplexer. Gefordert ist eine Top-Testbench mit kleinen Generics (TICK_CYCLES = 2, DEBOUNCE_CYCLES = 2, CLKS_PER_BIT = 8).

3 Programme und erwartetes Verhalten

Alle Programme wurden vorab mit einem Python-Modell auf Mikroschritt-Ebene ausgeführt (`05_pruefgrundlagen/mikroschritt_modell.py`). Zusammen nutzen sie alle elf Befehle.

SW3/SW2	Programm	RAM 0–15 (hex)	OUT
00	Fibonacci (Original)	`51 4E 50 E0 2E 4F 1E 4D 1F 4E 1D 70 63 00 00 00`	0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, dann Neustart
01	Zählen	`E0 2F 74 60 3F E0 80 64 00 00 00 00 00 00 00 01`	0 … 255, zurück bis 0, wieder hinauf
10	Multiplikation 3 × 5	`1E 3C 76 1D E0 F0 4E 1D 2F 4D 60 00 01 00 03 05`	15, dann HLT
11	Countdown mit NOP	`5A E0 00 3F 86 61 E0 F0 00 00 00 00 00 00 00 01`	10, 9, …, 1, 0, dann HLT

Vier Programme dienen als Hardwaretest über die UART (Antwort = alle OUT-Werte in Reihenfolge):

Test	Programm (hex)	Antwort (hex)	prüft
`fibonacci_halt`	`E02E7B4F1E4D1F4E1D6000F000000100`	`000101020305080d1522375990`	Fibonacci, HLT beim Überlauf (JC)
`multiply_7x9`	`1E3C761DE0F04E1D2F4D600001000709`	`3f`	7 × 9 durch Addition
`countdown_5`	`55E03F8561E0F000000000000000000001`	`050403020100`	SUB, JZ
`borrow_3_minus_5`	`533F75E0F051E0F00000000000000005`	`fe`	SUB mit Borgen, JC nicht genommen

4 Prüfgrundlagen

- **Mikroschritt-Modell** (Python) nach der Original-Tabelle; erzeugt die erwarteten Folgen.
- **Eigene Umsetzung** (eine Entity, `eigene_umsetzung_ben_eater_cpu.vhd`) als Gegenprobe der Aufgabe, der Referenz-Testbench und der Hardwarestrecke.
- **Referenz-Testbench** (dem System unbekannt): Programme „10“ und „11“ an den LEDs, Einzeltakt (genau ein Mikroschritt je Druck, kein Schritt durch einen 1-Takt-Impuls), die vier UART-Tests, Verwerfen einer ungültigen Länge, Löschen des RAM-Rests, keine UART-Ausgabe im Board-Modus, Rückkehr in den Board-Modus. Sie erkennt alle elf absichtlich eingebauten Fehler:

eingebauter Fehler	erkannt
JC prüft Z statt C	ja
SUB-Carry invertiert	ja
6 statt 5 Mikroschritte	ja
Lader löscht den RAM-Rest nicht	ja
UART-Ausgabe auch im Board-Modus	ja
HLT wirkt wie NOP	ja
keine Entprellung	ja
LED-Halbbytes vertauscht	ja
Länge 17 angenommen	ja
RI schreibt an Busadresse statt MAR-Adresse	ja
OUT wartet nicht auf den Sender	ja

5 Vorbereitung der Hardwarestrecke

Für den automatischen Hardwaretest musste die Kette PC → USB-UART → FPGA → zurück zum ersten Mal wirklich funktionieren. Der Abgleich mit der offiziellen Digilent-Master-XDC des Arty S7-50 und ein Probelauf deckten drei Fehler im System auf, die vorher nie aufgefallen waren, weil noch kein Lauf eine UART hatte:

#	Befund	Maßnahme
58	UART-Richtungen im Board-Wissen und im Skill vertauscht (Eingang ist V12, Ausgang R12); der XDC-Generator kannte keine UART-Pins (uart_rx ohne Pin → Vivado-Abbruch); Pmod-Listen mit Taster-, RGB- und Analog-Pins.	Wissen, Skill und Generator nach der Master-XDC korrigiert
59	Das System hätte seine eigene UART-Echo-Brücke in jeden Entwurf mit eigener UART eingebaut.	keine Brücke, wenn das Top-Level selbst UART-Ports hat
60	pyserial war keine Abhängigkeit und nicht installiert; der UART-Port war nicht gesetzt – der Hardwaretest konnte nie laufen.	Abhängigkeit, Konfiguration; Probelauf mit der eigenen Umsetzung: 4/4 am Board

6 Versuchsaufbau

- **MAS:** Server wie mit `start.py`, Aufgabe über `POST /api/chat` wie aus dem Dashboard, ohne Eingriff bis zum Ende einschließlich Flash und Hardwaretest (Stand `6f7f985`).
- **Single:** ein OpenCode-Aufruf über den Benchmark (`--variants single --opencode-timeout 3600`); Synthese, Flash und derselbe Hardwaretest danach mit dem FPGAAgent und dem HardwareTestAgent des Systems.
- Beide mit demselben Modell; je ein Lauf.

7 Ergebnisse

	MAS	Single
Dauer bis simulierter Entwurf	ca. 40 min	**8,4 min**
Dauer bis Hardwaretest bestanden	**45:44 min** in einem Lauf	ca. 14,5 min (Synthese, Flash, Test nachgezogen)
OpenCode-Aufrufe (Modellzeit)	27 (57,4 min)	1 (8,4 min)
Reparaturen	0	–
Module (12 vorgegeben)	12	12
Testbenches	12 je Modul + Top	1 (Top, 444 Zeilen)
Referenz-Testbench (neutrale Fassung)	bestanden	bestanden
Strenge Lesart „Verwerfen ab dem ersten Längenbyte“	nicht erfüllt	nicht erfüllt
Taktgenauer Vergleich mit dem Referenzmodell	bestanden (im Lauf)	4 von 4 Zufallsfolgen
Mutationsquote (10 Mutanten)	0,9	0,9
Hardwaretest am Board (4 UART-Programme)	**4/4, automatisch im Lauf**	4/4
Timing (WNS)	+3,10 ns	+3,01 ns
LUTs / Flipflops	399 / 508	428 / 479
Einzelanforderungen mit Nachweis	78, unabhängig bewertet	keine

7.1 Ablauf des MAS-Laufs



Abbildung 2: Abschnitte des MAS-Laufs in Minuten. Der Plan enthielt alle zwölf Module, 78 Einzelanforderungen, die vier Hardwaretests und eine Prüfanleitung; die UART-Brücke des Systems wurde übersprungen (#59).

7.2 Implementierung der Module

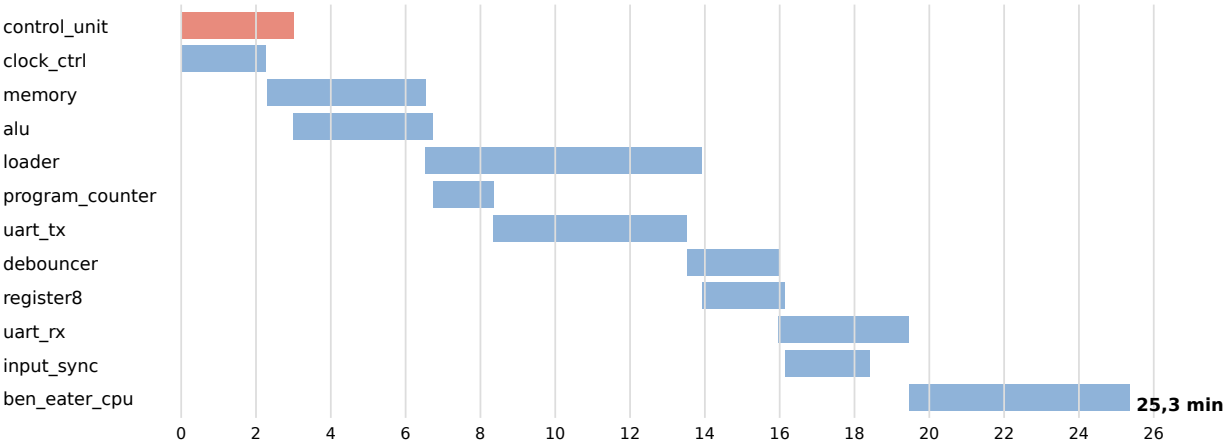


Abbildung 3: Modul und Testbench je Modul (Minuten ab Beginn der Implementierung, zwei Worker). Rot: das Steuerwerk `control_unit`. Alle Module außer dem Top-Level sind voneinander unabhängig; die Reihenfolge folgt

deshalb der Portzahl (#48), das Top-Level startet, sobald die übrigen elf fertig sind. Am längsten brauchte der Lader.

8 Unschärfe beim Verwerfen von Laderahmen

Die Aufgabe sagt: „Weicht ein Byte ab (Länge außerhalb 1..16, falsches Zeichen in START), wird der Rahmen verworfen und das nächste Byte wieder als erstes Längenbyte gelesen.“ Gemeint war eine byteweise Prüfung schon des ersten Längenbytes. **Beide** Entwürfe prüfen die Länge als 4-Byte-Wert erst nach dem vierten Byte:

```
              elsif len_cnt = 3 then
                if len_b1 = ZERO_BYTE and len_b2 = ZERO_BYTE
                   and unsigned(len_b0) >= to_unsigned(1, 8)
                   and unsigned(len_b0) <= to_unsigned(16, 8) then
```

Die ursprüngliche Referenz-Testbench sendete nur das Byte `0x11`; danach ging der folgende Rahmen bei beiden Entwürfen verloren. Da der Wortlaut beide Lesarten zulässt, wurde die Testbench vor der Auswertung von Single neutral gefasst: Sie sendet den vollständigen ungültigen Längenwert `11 00 00 00`, den beide Lesarten verwerfen müssen. Alle elf Fehlervarianten werden weiterhin erkannt. Bemerkenswert: Die unabhängige Abschlussprüfung des MAS bewertete die Anforderung als erfüllt, die MAS-Testbench prüfte genau den vollständigen 4-Byte-Fall – das System war in sich konsistent, die Aufgabe nicht eindeutig. Lehre für künftige Aufgaben: Fehlerverhalten von Protokollen mit einem Beispiel festlegen.

9 Einordnung

Korrektheit: Gleichstand, wie schon bei Aufgabe 04. Beide Entwürfe bestehen die Referenz-Testbench, den taktgenauen Modellvergleich und den Hardwaretest am Board; beide erreichen eine Mutationsquote von 0,9 und erfüllen das Timing mit ähnlichem Ressourcenbedarf (eigene Umsetzung zum Vergleich: 390 LUTs / 434 Flipflops, WNS +3,09 ns). **Aufwand:** Single war etwa fünfmal schneller bis zum simulierten Entwurf und brauchte ein Siebtel der Modellzeit. **Mehrwert des MAS:** der vollständige Weg bis zum bestandenen Hardwaretest in einem Lauf ohne Zusatzschritte, eine Testbench je Modul (Single: eine, wenn auch umfangreiche) und eine begründete Bewertung jeder Einzelanforderung.

> Über beide Aufgaben hinweg: Bei genau spezifizierten Aufgaben mit vorgegebener Modulstruktur löst ein einzelner Aufruf dieses Modells die Aufgabe ebenso korrekt und deutlich schneller. Das MAS gewinnt nicht bei der Lösungsquote, sondern bei Nachweisbarkeit und Durchgängigkeit. Ob es bei unschärfer spezifizierten oder größeren Aufgaben mehr leistet, ist offen; ein Lauf je Variante ist eine Stichprobe.

10 Grenzen

- Je ein Lauf je Variante; Ergebnisse freier Modelle streuen zwischen Läufen.
- Der Hardwaretest prüft nur gültige Rahmen und vier Programme; Fehlerverhalten des Laders und der Einzeltakt am Board sind nur in der Simulation geprüft.

- Die Single-Variante endet im Benchmark nach der Simulation; Synthese, Flash und Hardwaretest wurden mit den Werkzeugen des Systems nachgezogen.
- Die Aufgabe war beim Verwerfen von Laderahmen nicht eindeutig (Abschnitt 8).

11 Ablage und Reproduktion

Pfad	Inhalt
`Schaltungsagent/benchmark/prompts/05_ben_eater_8bit.md`	Aufgabe
`Schaltungsagent/benchmark/reference/05_ben_eater_8bit_ref_tb.vhd`	Referenz-Testbench
`Schaltungsagent/benchmark/reference/05_pruefgrundlagen/`	Mikroschritt-Modell, eigene Umsetzung, Gegenprobe
`docs/benchmark/ergebnisse/20261013-05_ben_eater_8bit/`	MAS- und Single-Entwurf, Berichte, Ereignisse, Log, Trajektorie, Vergleich
`docs/benchmark/ergebnisse/20261013-111444/`	Benchmark-Kennzahlen des Single-Laufs
`docs/benchmark/tools/aufgabe05_dokumentation.py`	erzeugt dieses Dokument

Literatur und Quellen

- B. Eater: Build an 8-bit computer from scratch (Video-Reihe und Unterlagen), eater.net/8bit; Mikrocode: github.com/beneater/eeprom-programmer, `microcode-eeprom-with-flags`.
- A. P. Malvino, J. A. Brown: Digital Computer Electronics, 3. Aufl., Glencoe/McGraw-Hill, 1993 (SAP-1).
- Digilent: Arty-S7-50-Master.xdc, github.com/Digilent/digilent-xdc.
- Y. Kim, X. Liu u. a. (Google Research): Towards a Science of Scaling Agent Systems. arXiv:2512.08296, 2025.